

BMCFuzz: Hybrid Verification of Processors by Synergistic Integration of Bounded Model Checking and Fuzzing

Shidong Shen^{1,2}, Jinyu Liu^{1,2}, Weizhi Feng^{1,2}, Fu Song^{1,2,3}, Zhilin Wu^{1,2}, 

¹ Institute of Software, Chinese Academy of Sciences, Beijing, China

² University of Chinese Academy of Sciences, Beijing, China

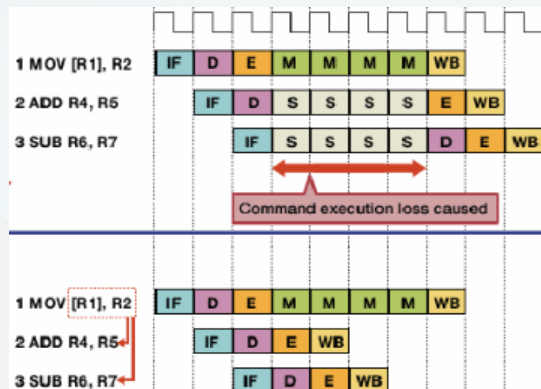
³ Nanjing Institute of Software Technology, Nanjing, China

{shensd,liujy25,fengwz,songfu,wuzl}@ios.ac.cn

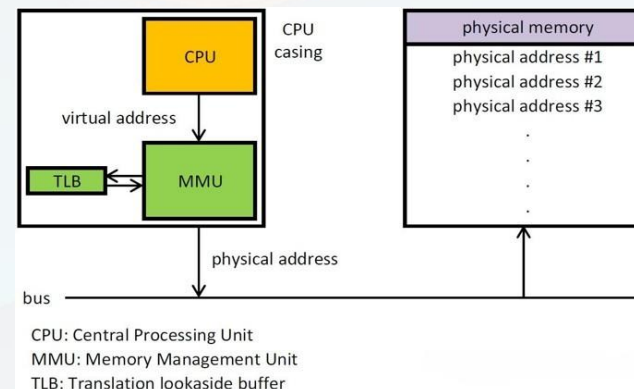
2025/10/28

Modern Processors

- Modern processors integrate advanced features to boost performance and security:
 - Out-of-order execution**
 - Virtual Memory**
 - Speculative execution**
- Design complexity grows rapidly → verification becomes increasingly **challenging**
- Early detection of design flaws in the pre-silicon stage is critical



Out-of-order execution



Virtual memory



1994 Intel FDIV problem
More than \$400 million
 to replace bad chips

Hardware Verification

Prior hardware verification approaches broadly fall into two categories:

Formal Verification

- Theorem proving
- Equivalence checking
- **Model checking** (Most commonly used for processor verification)

- ✓ Exhaustive space exploration
- ✓ Strong correctness guarantees
- ✗ Poor scalability on large designs

Simulation-Based Techniques

- Random testing
- Concolic/Symbolic execution
- **Hardware fuzzing**
 - Adapts from software fuzzing: mutate inputs → feedback-driven selection
 - Highly efficient

- ✓ Scalable and fast
- ✗ Incomplete exploration of the state space

Challenges

- **State-Space Explosion in Model Checking**
 - Exponential growth of states with register/memory bits and clock
 - **Bounded model checking (BMC)** limits path-depth but may miss deep bugs
- **Limited Coverage in Hardware Fuzzing**
 - Effectiveness hinges on seed diversity and feedback-driven selection
 - Hard to reach corner-case sequences in complex processor state spaces

**Can we combine both approaches to
get the best of both worlds?**

Related Work (Hardware Fuzzing)

Leverages software fuzzing techniques for hardware bug finding

- **Key Tools**

- RFUZZ / DiFuzzRTL / TheHuzz / XFUZZ / PathFuzz / ProcessorFuzz

- **Coverage Metrics**

- register, FSM, branch, instruction, CSR-transition coverages

- **Implementations**

- Adapting software fuzzers for hardware (RFUZZ, XFUZZ, PathFuzz)
- Fuzzing hardware as a software (Trippel et al.)
- Fuzzing hardware as hardware (TheHuzz, ProcessorFuzz, DiFuzzRTL)

- **Limitations**

- Coverage remains below industry standard
- Seed-diversity struggles to hit deep corner cases

Related Work (Model Checking)

- **General EDA Tools**

- Questa, VC Formal, JasperGold, SymbiYosys

- **RISC-V Processor Formal Verification**

- OneSpin, FormalISA, ChiRVFormal, riscv-formal
 - Compare reference model and Design Under Test (DUT) for model checking

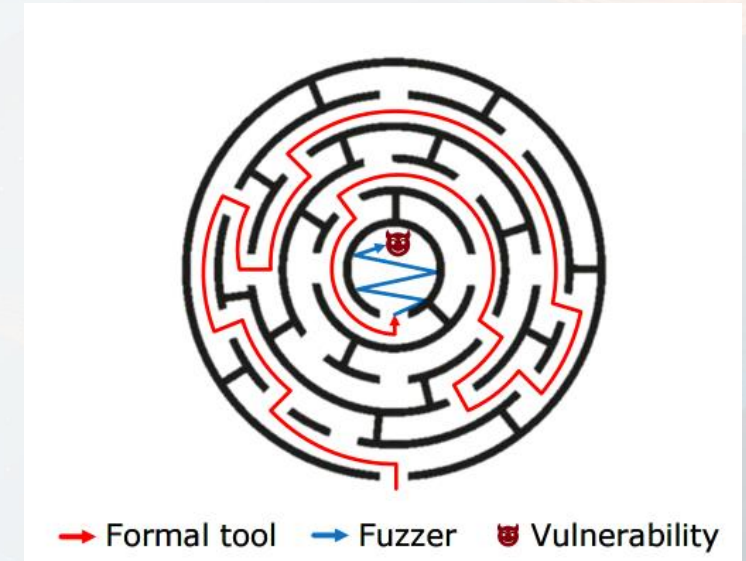
- **Limitations**

- State-space explosion limits full-design reach
- Writing SystemVerilog Assertions (SVAs) manually is time-consuming

Exhaustively verify whether a DUT satisfies specified properties.

Related Work (Hybrid Verification)

- **Prior Art**
 - **HyPFuzz**^[1]: Bounded model checking (BMC) for fuzzing seeds generation
 - **FormalFuzzer**: BMC preprocessing + cost-guided fuzzing
- **Shortcomings**
 - One-way data flow: no fuzzing → BMC feedback
 - Fixed initial states (HyPFuzz)
 - limited assertion templates (FormalFuzzer)
 - manual seed-space tuning (FormalFuzzer)



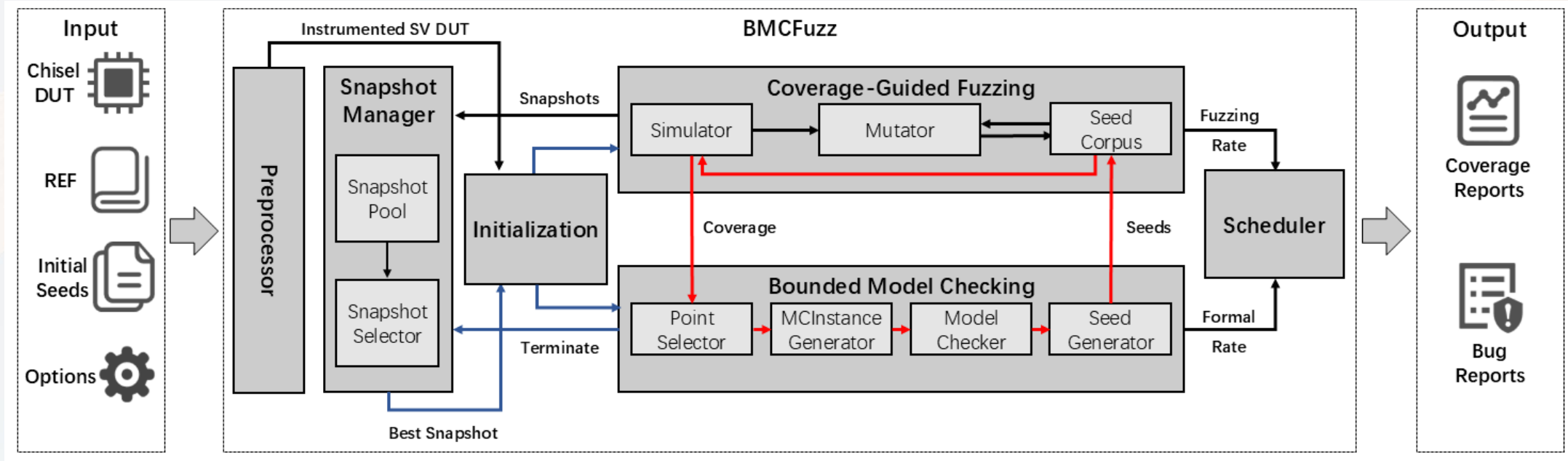
HyPFuzz method^[1]
(BMC- > Fuzz)

[1] Chen Chen, Rahul Kande, Nathan Nguyen, Flemming Andersen, Aakash Tyagi, Ahmad-Reza Sadeghi, and Jeyavijayan Rajendran. 2023. HyPFuzz: formal-assisted processor fuzzing. In *Proceedings of the 32nd USENIX Conference on Security Symposium (SEC '23)*. USENIX Association, USA, Article 77, 1361–1376.

Contributions

- **Snapshots for “Fuzzing→BMC”:**
 - Capture circuit-level register states whenever privilege levels or key CSR bits change
 - Use snapshots as initial states for deeper BMC exploration
- **Priority-Based Snapshot Scoring:**
 - Score and pick the “best” snapshots to feed into BMC
 - Dynamically adjust scores to maintain diversity and efficiency
- **Memory Abstraction for BMC:**
 - Record only the actually accessed addresses
 - Cache-style updates maintain memory consistency
 - Greatly reduces state space, improving BMC depth and efficiency

Overview



Our BMCFuzz framework (blue: snapshot loop; red: BMC-CGF loop)

Synergistic Integration of BMC and Fuzzing Multiple bug discovery paths

- BMC finds the uncovered key points and generates seeds for fuzzing;
- Fuzzing produces snapshots used as initial states for BMC.
- BMC -> Seeds -> Fuzzing -> Bug (**Old HypFuzz method**)
- BMC -> Seeds -> Fuzzing -> Snapshot -> BMC -> Bug
- Fuzzing -> Snapshot -> BMC -> Seeds -> Fuzzing -> Bug
- ...

Snapshot Definition

**Snapshot: Complete circuit-level register states
(architectural + microarchitectural)**

- Architectural Registers :
 - General Registers(x0-31), Program Counter (PC), Control and status registers (CSRs), etc.
- Microarchitectural Registers:
 - pipeline registers, branch predictor status, Reorder Buffer (ROB), etc.
- Memory Subsystem Snapshot:
 - includes L1/L2 cache states if necessary

Snapshot Roles

- **Two-Way Feedback Loop**
 - Fuzzing → snapshot capture → BMC uses snapshots as seeds
→ new seeds fed back into Fuzzing
- **Multi-Starting Point Exploration**
 - Provide multiple valid start states to expand state coverage
- **Reality-Guaranteed States**
 - Derived from actual simulator runs
 - Prevents “fake covered point” caused by unrealistic register values
- **Skip Redundant Initialization**
 - Avoid complex start-up logic

Snapshot Capture Conditions

- **Only recorded** at clock cycles where
 - Privilege level **changes**
 - Any monitored Control and Status Registers (CSRs) field **changes**

CSR	Field(s)	Description
mstatus	MIE / SIE / UIE	Global interrupt enables
	MPIE / SPIE / UPIE	Saved interrupt-enable bits before trap
	MPP / SPP	Privilege mode prior to trap
	FS / XS	Floating-point unit & user-extension state
	MPRV	Memory-access privilege control
	SUM / MXR / TVM / TW / TSR	Access and trap control
	MXL / SXL / SD	Integer-register width config & context-switch state
satp	MODE / ASID / PPN	Virtual-address translation mode, ASID, and physical page number
medeleg	–	Exception delegation bits from machine to supervisor mode

Key CSRs and Their Fields

Snapshot Selection

All the monitored fields are partitioned to five selection criteria for **RISC-V** processors:

- C_1 : *Privilege Mode*
 - $G_1 = \{\text{Privilege level}\}$
- C_2 : *Virtual Memory*
 - $G_2 = \{\text{the MXR, SUM, MPRV, MODE fields of mstatus}\}$
- C_3 : *Specific Illegal-Instruction Exception*
 - $G_3 = \{\text{the TVM, TW, TSR fields of mstatus}\}$
- C_4 : *Trap*
 - $G_4 = \{\text{the MPP, SPP, MPIE, SPIE, MIE, SIE fields of mstatus}\}$
- C_5 : *Exception delegation*
 - $G_5 = \{\text{all fields of medelegs}\}$

Score Computation:

$$\text{Score}(s) = \sum \{W_i \mid \text{Tags}(s) \cap G_i \neq \emptyset\}.$$

Weight Decay:

$$W_i := \begin{cases} W_i \times D, & \text{if } \text{Tags}(s) \cap G_i \neq \emptyset; \\ W_i, & \text{otherwise.} \end{cases}$$

Coverage-Guided Fuzzing (CGF)

- **Fuzzer**
 - PATHFUZZ (Built on LibAFL) + DiffTest
- **Mutation Engine**
 - LibAFL mutator: bit-flip, byte-flip, clone, swap
 - Chooses mutations based on coverage improvement
- **Simulator Workflow**
 - DUT execution via Verilator (commit buffered instructions)
 - Cosimulate DUT <-> Spike REF in DiffTest
 - Compare instruction commit, load/store, exceptions, interrupts, CSRs
 - On mismatch → report potential bug
 - Record snapshots & update coverage

Bounded Model Checking (BMC)

- **Point Selector**

- Picks batches of uncovered points (least-covered submodules first)
- Update uncovered points based on CGF coverage information

- **MC Instance Generator**

- Inserts **SystemVerilog Assertions (SVA)** for each target point
- Applies lightweight memory abstraction (cache-like) to trim state space

```
cover property (@(posedge clk)
    tltoaxi4.counter_0[0] === 1'b1);
```

SVA cover property

Bounded Model Checking (BMC)

- **Model Checker**

- Two options: **SAT-based BMC (bit-level)** vs. **SMT-based BMC (word-level)**
 - SAT-based BMC run faster after bit-blasting;
 - SMT-based BMC keeps word-level structure for full memories
- Bit-blasting full **CPU memory is infeasible** → introduce a **memory abstraction** (next)
- SAT-based BMC solves a Reg-tog cover point in Rocket in **2 min**, compared with **20 min** for SMT-based BMC
- Uses SAT-based BMC **with memory abstraction** for efficient verification

- **Seed Generator**

- Replays BMC witness on DUT to reconstruct a new fuzzing seed
- Bridges abstract witness → concrete seed for the fuzzer

Bounded Model Checking (BMC)

- **Memory Abstraction**

- Full-scale SoC memory (GiB-level) → exponential BMC state explosion
- Cache-Like structure, only record actually accessed addresses
- Consistency Rules
 - First access to an address → return any valid value, fill cache line
 - Subsequent reads (no write) → return first value
 - Write → update cache line, future accesses return new value

Parameter	Original Memory	Abstract Memory
Address Space	2 GiB	2 GiB
Memory Lines	268,435,456	16
Data Width per Line	64 bits	64 bits
Total Bits	2^{34}	$16 \times (1 + 28 + 64) = 1504$
BMC Reachable Instructions Count	8	16

Memory Abstraction Example (Rocket)

Scheduler

Scheduler: Switching Between CGF and BMC in the inner loop.

Fuzzing rate:

- Moving average **“new coverage points/sec”** over the last ≤ 100 seeds in CGF.
- Shows how fast fuzzing is discovering new points.

Formal rate:

- Moving average **“resolved uncovered points/sec”** in BMC.
- Shows how fast BMC is solving coverage gaps.

Switching rule:

- If fuzzing rate $<$ formal rate $\rightarrow$ switch to BMC.
- After trying to cover a bounded number of uncovered points by BMC, then switch to CGF.

Evaluation

Comparison Criteria:

- **Coverage:** Ability to explore the state space of the design

Case Studies: NutShell, Rocket and BOOM

Name	#LoC	Pipeline	Single Issue	OoO	Sv39	TLB	Privilege	BP	Cache
NutShell	7,220	9 stages	✓	✗	✓	✓	M, S, U	✓	L1
Rocket	11,542	5 stages	✓	✗	✓	✓	M, S, U	✓	L1
BOOM	13,123	7 stages	✓	✓	✓	✓	M, S, U	✗	L1

Characteristics of three RISC-V processors, where
BP denotes Branch Prediction, TLB denotes Translation Lookaside Buffer.

Experimental Setup

- Ubuntu 22.04
- 2048 GiB RAM and 256 Intel(R) Xeon Platinum 8153 CPUs (2.00 GHz)
- Golden Reference Model: Spike (in C++)
- BMC Model Checker: rIC3
- Initial Seeds: riscv-tests (140 seeds) + riscv-dv (1150 seeds)

Processor	Coverage	Timeout(h)	BMC timeout (h)	BMC step
NutShell	Reg-tog	24	2	90
	Branch	12	2	90
	Mux-tog	12	2	90
Rocket	Reg-tog	24	3	75
	Branch	12	3	75
	Mux-tog	12	3	75
BOOM	Reg-tog	48	4	75
	Branch	24	4	75
	Mux-tog	24	4	75

Time limit and BMC step bound

Coverage Evaluation

BMCFuzz achieves the best coverage in all the combinations of three processors and three coverage metrics, especially:

- Compared to PATHFUZZ: **5.01%** (Reg-tog), **1.71%** (Branch), **1.63%** (Mux-tog) on NutShell
- Compared to HyPFuzz* ¹: **12.08%** (Reg-tog), **15.28%** (Branch), and **8.32%** (Mux-tog) on BOOM

TABLE III: Experimental results, where $x(a, b)$ in the BMCFuzz column denotes that the coverage of BMCFuzz is $x\%$, which is $a\%$ (resp. $b\%$) more than that of PATHFUZZ (resp. HyPFuzz*), and [†] denotes we use linear memory instead of footprint memory.

DUT	Coverage	Coverage Points	Symbiosys+rIC3(%)	PATHFUZZ (%)	HyPFuzz*(%)	BMCFuzz(%)
NutShell	Reg-tog	13,003	31.77	84.77	89.38	89.78(5.01, 0.40)
	Branch	1,939	69.73	94.22	95.36	95.93(1.71, 0.57)
	Mux-tog	1,349	48.78	93.63	94.89	95.26(1.63, 0.37)
Rocket	Reg-tog	8,744	24.10	85.58	78.95	86.78(1.20, 7.83)
	Branch	1,358	37.41	70.99	68.19	71.06(0.07, 2.87)
	Mux-tog	1,306	32.85	76.88	73.12	77.41(0.53, 4.29)
BOOM	Reg-tog	19,229	22.19 [†]	83.91 [†]	72.63 [†]	84.71(0.80, 12.08)[†]
	Branch	12,966	58.37 [†]	92.22 [†]	78.52 [†]	93.80(1.58, 15.28)[†]
	Mux-tog	4,629	41.89 [†]	83.43 [†]	80.19 [†]	88.51(5.08, 8.32)[†]

¹ HyPFuzz* refers to our reimplement of HyPFuzz, since the original tool is not open-source. Its workflow corresponds to the inner loop of BMCFuzz that uses the reset state as the snapshot.

Coverage Evaluation

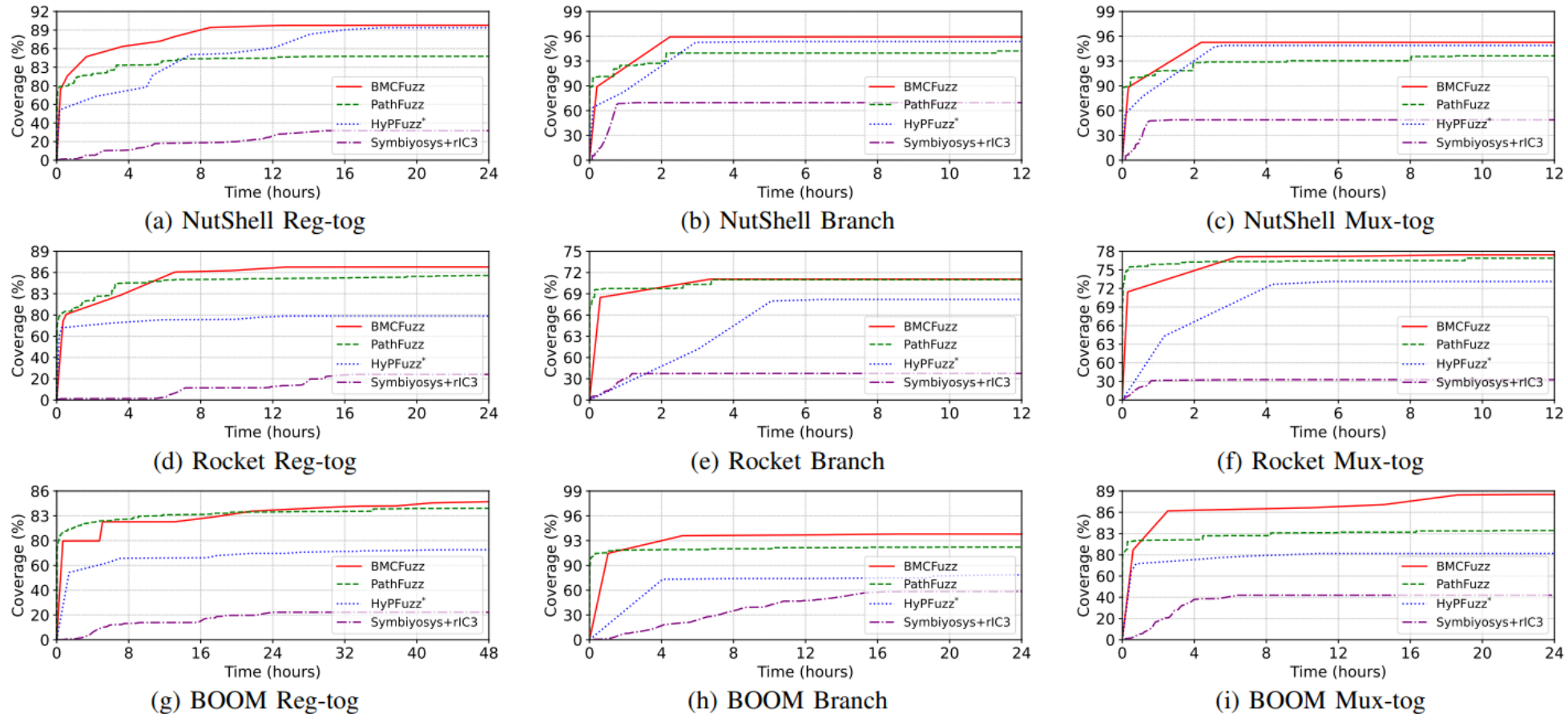


Fig. 2: Coverage curves of different tools

As design size increases from NutShell to Rocket to BOOM, HyPFuzz* slows down, while our BMCFuzz scales more efficiently and maintains a lead.

Bug Detection

1. **NutShell:N1.** The reserved D, A, and U bits of non-leaf page table entries (PTEs) may not be zeroed, inconsistent with the RISC-V specification and the reference model Spike
2. **NutShell:N2.** When the SV39 virtual memory is enabled, a TLB exception causes NutShell to hang due to a hardcoded condition that disallows the required exception propagation
3. **NutShell:N3.** Illegal read/write mask of the control and status register medeleg which determines the delegation of execution handling (a wrong mask 0xbbff is used instead of the correct one 0xb3ff)
4. **BOOM:B1.** When Svnapot is not implemented and the reserved 63-th bit of PTEs is not zeroed, the pagefault exception did not raise, inconsistent with the RISC-V specification and the reference model Spike

Conclusion

- **BMCFuzz Overview**

- Introduced **Snapshots** for hardware fuzzing → BMC seeding
- First **two-way hybrid** integration of BMC $\leftrightarrow$ CGF

- **Key Achievements**

- **Improved coverage** on NutShell, Rocket, BOOM
- **Discovered** 4 previously unknown bugs
- **Released** an **Open-source tool** for hybrid hardware verification

- **Future Work**

- Integrate with non-DiffTest CGF engines (e.g., Cascade, DiFuzzRTL)
- Optimize performance for deeper model checking
- Apply to **more complex cores** (e.g., XiangShan)

Thanks!

Open source repo:

<https://github.com/iscas-versys/BMCFuzz>



GitHub Repo



Our Lab Website